

秒杀系统与 AI 智能客服

项目概述

文档性质：方案讨论稿

用于确认项目定位、功能边界、建设顺序与部署方向

当前判断：秒杀业务底座已形成；AI 客服与生产部署待建设

版本 V0.1 · 2026 年 8 月 12 日

目录

- 目录.....2
- 1. 项目定位.....4
 - 1.1 建设目标.....4
 - 1.2 项目价值.....4
 - 1.3 文档边界.....4
- 2. 当前建设状态4
 - 2.1 当前已有技术基础5
 - 2.2 需要特别说明的事实5
- 3. 总体功能范围6
 - 3.1 秒杀商城功能.....6
 - 3.2 AI 智能客服功能（建议第一版）6
- 4. 总体技术架构6
 - 4.1 服务划分7
 - 4.2 为什么 AI 客服建议独立成服务8
- 5. 秒杀高并发核心设计.....8
 - 5.1 关键机制.....8
 - 5.2 性能目标的正确表达9
- 6. AI 智能客服方案.....9
 - 6.1 推荐工作流程.....9
 - 6.2 建议能力分级.....9
 - 6.3 安全边界.....10
 - 6.4 模型选择建议.....10
- 7. 阿里云 ECS 部署方案10
 - 7.1 第一版资源建议10
 - 7.2 MinIO 上云要点11
 - 7.3 安全组与网络.....11
 - 7.4 后续扩容方向.....11

8. 当前缺口与风险 11

 8.1 范围控制建议..... 12

9. 推荐建设路线 12

 9.1 建议下一步..... 13

10. 项目验收建议..... 13

结论..... 13

1. 项目定位

本项目拟建设一个“高并发秒杀商城 + AI 智能客服”综合系统。系统一方面通过 Redis、Lua 脚本、RabbitMQ 和微服务架构处理秒杀场景中的瞬时流量、库存一致性与异步订单问题；另一方面通过大模型与业务知识库，为用户提供商品、活动、订单及平台规则相关的智能问答。最终目标是以 Docker 化方式部署到阿里云 ECS。

1.1 建设目标

- 完成一个可以注册、登录、浏览商品、参与秒杀、查询订单的完整业务闭环。
- 展示限流思想、Redis 原子扣库存、消息队列削峰、幂等与补偿等高并发设计。
- 增加 AI 智能客服，使用户可以用自然语言查询平台规则、活动信息和本人订单状态。
- 形成可在一台阿里云 ECS 上运行的第一版部署方案，并保留后续横向扩容能力。

1.2 项目价值

价值维度	说明
业务完整性	覆盖用户、商品、秒杀、订单、客服与后台管理，不只是单独的抢购接口。
技术展示性	同时体现 Spring Cloud 微服务、高并发处理、异步消息、对象存储和 AI 应用集成。
演进能力	第一版可单机部署，后续可以将热点服务扩展到多台 ECS，并迁移至云数据库、云 Redis 与 OSS。

1.3 文档边界

本文是一份用于确认项目方向的概述，不等同于最终需求规格、详细设计或部署手册。所有“规划”内容需要在后续开发、测试通过后，才能改写为“已实现”。

2. 当前建设状态

根据当前代码仓库核对，项目已经具备秒杀商城的主要后端模块与 Vue 前端页面，但 AI 客服和完整的生产部署尚未落地。

模块	范围	当前判断
用户与安全	注册、密码/验证码登录、JWT、网关鉴权	已实现 代码中已有完整入口与基础安全机制
商品与图片	商品管理、批量上传、MinIO 对象存储	已实现 需要在上云前拆分 MinIO 内部地址与公开地址
秒杀核心	活动、动态路径、Redis Lua 原子扣库存	已实现 已具备防超卖与一人一单基础
订单链路	RabbitMQ 异步建单、结果查询、失败补偿	已实现 已有消息幂等与补偿思路
AI 智能客服	对话、知识库检索、订单/活动工具调用	待建设 仓库中暂未发现 AI 客服服务与前端会话界面
生产部署	Docker 化、Nginx、HTTPS、监控、备份	待建设 当前 Compose 只覆盖基础设施

2.1 当前已有技术基础

- 后端：Java 17、Spring Boot 3.2、Spring Cloud 2023、Spring Cloud Alibaba。
- 注册与路由：Nacos、Spring Cloud Gateway、OpenFeign。
- 数据与中间件：MySQL、Redis、RabbitMQ、MinIO。
- 前端：Vue 3、Vite、Axios、Pinia、Vue Router。
- 压测：仓库中已有 JMeter 测试资料，可用于后续建立单机性能基线。

2.2 需要特别说明的事实

“AI 智能客服”目前属于项目目标，而不是现有能力。正式展示或答辩时，应明确说明完成度，避免把方案设计误写成实际功能。

3. 总体功能范围

用户角色	主要功能
普通用户	注册与登录、查看商品和秒杀活动、获取秒杀路径、提交抢购、查询抢购结果、查看/支付/取消订单、咨询 AI 客服。
管理员	维护商品与图片、配置秒杀活动和库存、执行缓存预热、查看全部订单、维护客服知识内容（规划）。
系统服务	网关鉴权与路由、库存原子扣减、消息排队、异步建单、失败补偿、日志与健康检查。

3.1 秒杀商城功能

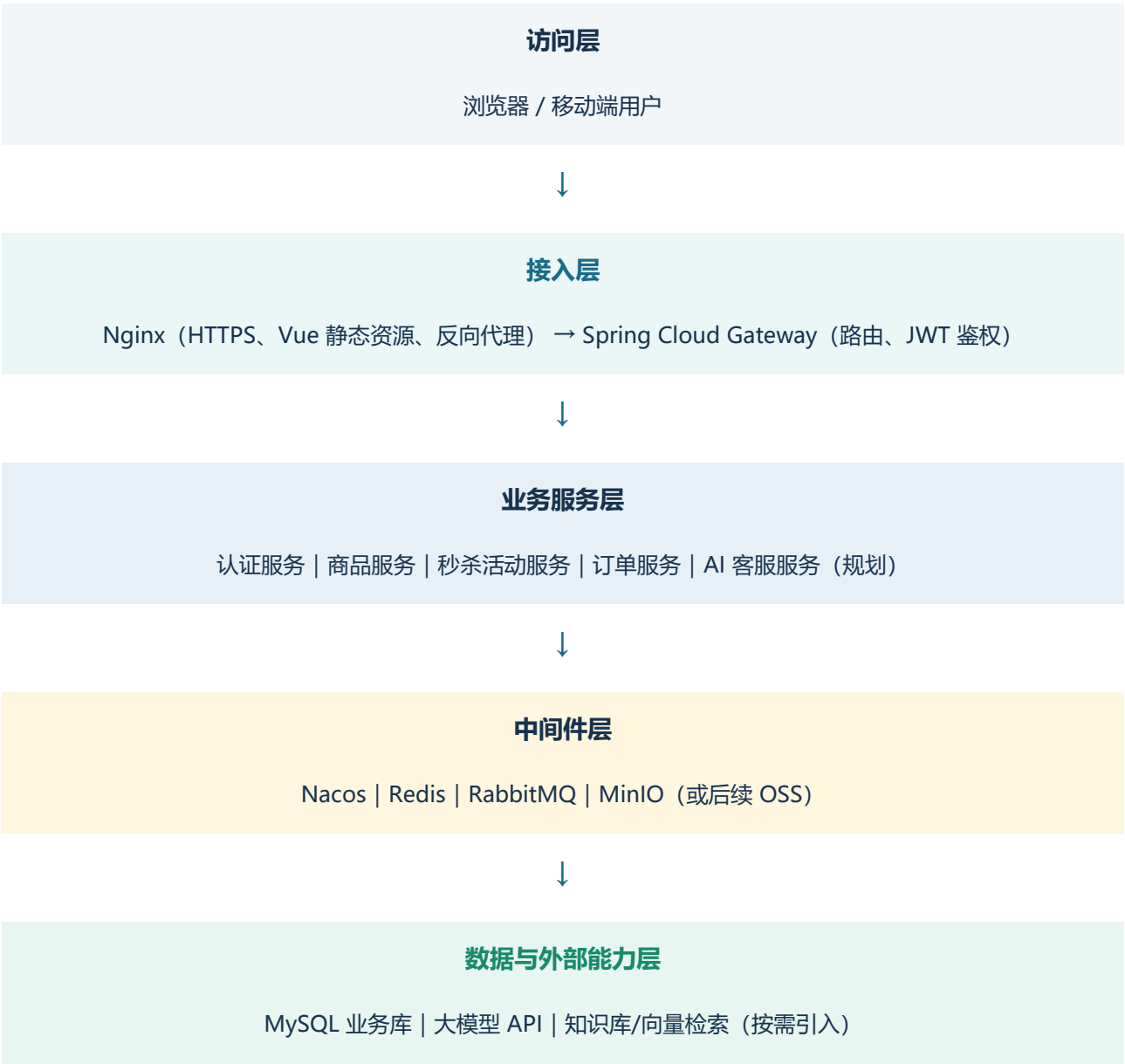
- 账户体系：密码登录、邮箱验证码登录、注册、重置密码、刷新令牌与退出。
- 商品体系：商品信息维护、多图片上传、MinIO 存储、商品展示。
- 活动体系：活动时间、预告、秒杀商品、活动库存、每人限购。
- 抢购体系：动态秒杀路径、Redis 库存、重复购买判断、排队结果查询。
- 订单体系：异步建单、个人订单、支付、取消、管理员查询。

3.2 AI 智能客服功能（建议第一版）

- 回答平台规则：如何注册、秒杀流程、支付规则、订单取消、常见异常。
- 回答实时活动问题：当前有什么活动、某商品何时开抢、库存是否已售罄。
- 查询本人业务数据：我的订单在哪里、抢购是否成功、订单当前状态。
- 无法确认时明确提示，并提供转人工或反馈入口；不编造库存、价格和订单状态。

4. 总体技术架构

第一版建议保持现有 Spring Cloud 结构，并新增独立的 AI 客服服务。所有公网请求统一经过 Nginx 和 Gateway，业务服务及中间件不直接暴露到互联网。



4.1 服务划分

服务	主要职责	状态
Gateway	统一入口、路由、JWT 校验、管理员鉴权、用户信息透传。	已实现
Auth Service	用户、验证码、登录、注册、刷新令牌。	已实现
Product Service	商品管理、MinIO 图片上传与删除。	已实现
Activity Service	秒杀活动、动态路径、Redis Lua 扣库存。	已实现
Order Service	RabbitMQ 消费、异步建单、幂等与补偿。	已实现

服务	主要职责	状态
AI Customer Service	会话管理、知识检索、大模型调用、受控业务工具。	规划新增

4.2 为什么 AI 客服建议独立成服务

- AI 请求耗时、计费方式和失败模式与秒杀接口不同，独立部署更容易限流与隔离。
- 以后切换大模型供应商、增加知识库或调整提示词时，不影响秒杀核心链路。
- 客服服务只能通过受控接口查询订单和活动，不能让大模型直接连接数据库或修改库存。

5. 秒杀高并发核心设计

高并发并不是单靠“多启动几个 Java 程序”完成，而是通过入口保护、缓存原子操作、消息队列和数据库兜底共同完成。当前代码已经覆盖其中的关键基础。



5.1 关键机制

机制	解决的问题	当前情况
动态秒杀路径	降低接口被脚本直接反复调用的风险。	已实现，路径写入 Redis 并设置短期有效。
Redis + Lua	将检查库存、扣库存、重复购买判断作为一个原子操作，降低超卖风险。	已实现。
RabbitMQ	将瞬时抢购请求转为排队消息，避免所有成功请求同时写 MySQL。	已实现。
幂等与补偿	处理消息重复投递、订单创建失败和库存恢复。	已有 eventId 与补偿事件思路。
入口限流	超出系统能力时快速拒绝，保护 Redis、MQ 与数据库。	建议补充 Gateway/Redis 限流。

机制	解决的问题	当前情况
数据库约束	作为重复订单和异常情况的最后一道防线。	需要结合 SQL 约束继续验收。

5.2 性能目标的正确表达

在未完成统一环境压测前，不建议直接宣称“支持十万并发”。应先记录 ECS 配置、线程数、测试数据、成功率和响应时间，再给出可复现结论，例如：“在 4 核 8G 测试环境下，秒杀入口达到 X 请求/秒，错误率低于 Y%。”

6. AI 智能客服方案

建议先做“可控、能解释、能演示”的第一版：大模型负责理解问题和组织答案，平台知识库提供规则依据，业务工具提供实时数据。不要一开始就建设复杂的本地大模型或大型向量数据库。

6.1 推荐工作流程

1. 用户在网页右下角打开客服窗口并发送问题。
2. Gateway 完成身份校验，将问题和用户身份传给 AI 客服服务。
3. 客服服务判断问题类型：平台知识、活动商品、本人订单或闲聊。
4. 平台知识从知识库检索；实时数据通过内部只读接口查询；不把数据库账号交给大模型。
5. 将检索结果、业务数据和安全规则组合后调用大模型。
6. 输出答案并保存必要的会话记录；敏感信息脱敏，异常时给出明确兜底回复。

6.2 建议能力分级

阶段	能力	建议实现
MVP	FAQ + 流式对话	整理平台规则文档；调用云端大模型 API；实现客服浮窗与会话接口。
增强版	实时业务查询	为活动、商品和个人订单提供受控的只读工具接口，并进行用户身份校验。

阶段	能力	建议实现
扩展版	RAG 与运营后台	知识文档切分、向量检索、引用来源、知识更新、会话评价与人工转接。

6.3 安全边界

- AI 不能执行扣库存、改价格、支付、退款等高风险操作。
- 查询订单必须以 Gateway 认证后的 userId 为准，不能相信用户在问题中提供的用户编号。
- 大模型 API Key 只放在服务端环境变量中，不写入 Vue 前端或 Git 仓库。
- 限制单用户请求频率、上下文长度与每日额度，防止接口滥用和费用失控。
- 对于库存、价格、活动时间和订单状态，必须以业务接口返回的数据为准。

6.4 模型选择建议

项目最终部署在阿里云 ECS，第一版可优先考虑阿里云百炼/通义系列的 API 服务，以降低网络和运维复杂度；也可以保留统一的模型接口，后续切换其他兼容服务。具体模型和费用应在开发时根据最新官方价格、上下文长度及并发限制重新确认。

7. 阿里云 ECS 部署方案

第一阶段以“低成本、能上线、易维护”为目标：使用一台 ECS 运行完整系统，通过 Docker Compose 组织服务。该方案适合学习、演示和小流量访问，但不是高可用生产集群。

7.1 第一版资源建议

资源	建议	说明
ECS	建议 4 核 8G 起步	同机运行多个 Java 服务及 MySQL、Redis、RabbitMQ、Nacos、MinIO。预算紧张可先短期测试。
云盘	系统盘 + 独立数据盘	MySQL、MinIO 等持久数据落在数据盘，并配置快照。
域名与 HTTPS	Nginx 统一入口	公网仅开放 80/443；SSH 端口限制来源 IP。

资源	建议	说明
对象存储	第一版 MinIO，后续可迁移 OSS	MinIO 数据必须挂载到云盘，不能只存在容器层。
AI 模型	优先调用云端 API	避免在单台 4 核 8G ECS 上运行大型本地模型。

7.2 MinIO 上云要点

- Java 服务内部连接地址使用 Docker 服务名，例如 `http://minio:9000`。
- 浏览器公开图片地址使用域名，例如 `https://example.com/images/...`，由 Nginx 转发。
- 程序配置应拆分 `MINIO_ENDPOINT` 与 `MINIO_PUBLIC_URL`；数据库优先保存对象名或相对路径。
- 图片目录挂载到 ECS 数据云盘，并配置云盘快照或定期同步到 OSS。

7.3 安全组与网络

公网建议只开放 80、443，以及限制来源 IP 的 SSH 端口。MySQL、Redis、RabbitMQ、Nacos、MinIO API、Gateway 和各业务服务端口应放在 Docker 内部网络，不直接暴露公网。

7.4 后续扩容方向

当单台 ECS 出现 CPU、内存或中间件瓶颈时，再将 Gateway、Activity Service、Order Service 扩展到多台 ECS，并把 MySQL、Redis、对象存储逐步迁移到阿里云托管产品。是否扩容应由监控和压测数据决定。

8. 当前缺口与风险

优先级	缺口/风险	建议处理
P0	AI 客服尚未实现	先确定 MVP 边界，再新增独立客服服务、会话接口和前端浮窗。
P0	业务服务未完整 Docker 化	为 5 个现有服务及新增客服服务提供 Dockerfile，补齐生产 Compose。

优先级	缺口/风险	建议处理
P0	MinIO 内外地址未分离	增加公开 URL 配置，避免 ECS 上出现上传成功但浏览器无法显示。
P0	生产密钥与默认密码	全部改为环境变量；RSA 私钥、邮箱授权码、模型 API Key 不进入仓库。
P1	入口限流与防刷不足	增加 Gateway/Redis 限流、用户/IP 维度限制和异常请求监控。
P1	监控、日志、备份不足	增加统一日志、健康检查、资源监控、RabbitMQ 积压监控和数据备份。
P1	缺少可复现性能结论	在固定 ECS 规格和固定数据集上执行 JMeter 阶梯压测并保存报告。
P2	单台 ECS 存在单点故障	第一版接受该限制；项目说明中明确，后续再设计多机高可用。

8.1 范围控制建议

为了保证项目真正完成，第一版 AI 客服应聚焦“问答 + 只读查询”，暂不加入自动退款、自动修改订单、多智能体协作、本地大模型训练等高风险或高成本能力。

9. 推荐建设路线

里程碑	主要工作	完成标志
M1：秒杀底座验收	梳理现有功能、补自动化测试、核对库存与订单一致性。	注册到订单闭环可重复演示；关键异常有测试结果。
M2：AI 客服 MVP	新增客服服务、模型 API、FAQ 知识、流式对话页面。	能稳定回答平台规则，并对未知问题正确兜底。
M3：业务工具接入	增加活动、商品、本人订单只读查询工具与权限校验。	用户可自然语言查询真实业务数据，且无法越权。

里程碑	主要工作	完成标志
M4: 容器化部署	Dockerfile、生产 Compose、Nginx、MinIO 公网地址、环境变量。	本机可用一条 Compose 命令启动完整系统。
M5: ECS 上线	域名、HTTPS、安全组、数据盘、快照、监控与日志。	公网可访问，重启容器后业务数据与图片不丢失。
M6: 压测与总结	JMeter 阶梯压测、定位瓶颈、记录配置与结论。	形成可复现性能报告和项目答辩材料。

9.1 建议下一步

先不要立即购买 ECS。建议首先确认本文中的项目范围，然后设计 AI 客服 MVP 的接口与数据边界；与此同时补齐 MinIO 内外地址设计。范围确认后，再进入客服实现和完整 Docker 化。

10. 项目验收建议

验收维度	最低验收标准
业务闭环	普通用户可以完成注册/登录、浏览活动、秒杀、查询结果和查看订单。
并发正确性	库存不出现负数；同一用户不能重复购买；重复消息不产生重复订单。
AI 客服	FAQ 回答准确；实时查询来自业务接口；无权限时拒绝；模型异常时有兜底。
部署	一条 Compose 命令可启动；仅入口端口暴露公网；HTTPS 可用。
数据可靠性	重启容器后数据库与图片仍存在；有快照或备份恢复说明。
可观测性	能查看服务健康、错误日志、CPU/内存及 RabbitMQ 消息积压。
性能报告	说明测试机器、并发模型、吞吐量、响应时间、成功率和瓶颈，不使用无依据的宣传数字。

结论

这个项目具备继续完善的价值。 现有秒杀微服务已经提供了较好的技术底座，下一阶段最重要的不是继续堆叠更多中间件，而是把 AI 客服做成一个边界清楚、数据可信、安全可控的真实功能，并完成可重复

的容器化部署与测试。

建议将本稿作为项目范围讨论材料使用；确认功能边界后，再扩展为正式的需求说明书、详细设计文档和 ECS 部署手册。